

MODULE 5

TRANSACTION PROCESSING

Dr. Geetha Mary A
Professor,
SCOPE, VIT, Vellore

Introduction to Transaction Processing

- **Single-User System:** At most one user at a time can use the system.
- **Multuser System:** Many users can access the system concurrently.
- **Concurrency**
 - ▣ **Interleaved processing:** concurrent execution of processes is interleaved in a single CPU
 - ▣ **Parallel processing:** processes are concurrently executed in multiple CPUs.

Introduction to Transaction Processing

- **A Transaction:** logical unit of database processing that includes one or more access operations (read -retrieval, write - insert or update, delete).
- **A transaction (set of operations)** may be stand-alone specified in a high level language like SQL submitted interactively, or may be embedded within a program.
- **Transaction boundaries:** Begin and End transaction.
- An **application program** may contain several transactions separated by the Begin and End transaction boundaries.

Introduction to Transaction Processing

- **A database** - collection of named data items
- **Granularity of data** - a field, a record , or a whole disk block
(Concepts are independent of granularity)
- Basic operations are **read** and **write**
 - **read_item(X)**: Reads a database item named X into a program variable. To simplify our notation, we assume that *the program variable is also named X*.
 - **write_item(X)**: Writes the value of program variable X into the database item named X.

Introduction to Transaction Processing

READ AND WRITE OPERATIONS:

- Block Transfer
- **read_item(X) command includes the following steps:**
 1. Find the address of the disk block that contains item X.
 2. Copy that disk block into a buffer in main memory.
 3. Copy item X from the buffer to the program variable named X.

Introduction to Transaction Processing

- **write_item(X) command includes the following steps:**
 1. Find the address of the disk block that contains item X.
 2. Copy that disk block into a buffer in main memory (if that disk block is not already in some main memory buffer).
 3. Copy item X from the program variable named X into its correct location in the buffer.
 4. Store the updated block from the buffer back to disk (either immediately or at some later point in time).

Two sample transactions. (a) Transaction T_1 . (b) Transaction T_2 .

(a) T_1

read_item (X);
 $X := X - N$;
write_item (X);
read_item (Y);
 $Y := Y + N$;
write_item (Y);

(b) T_2

read_item (X);
 $X := X + M$;
write_item (X);

Introduction to Transaction Processing

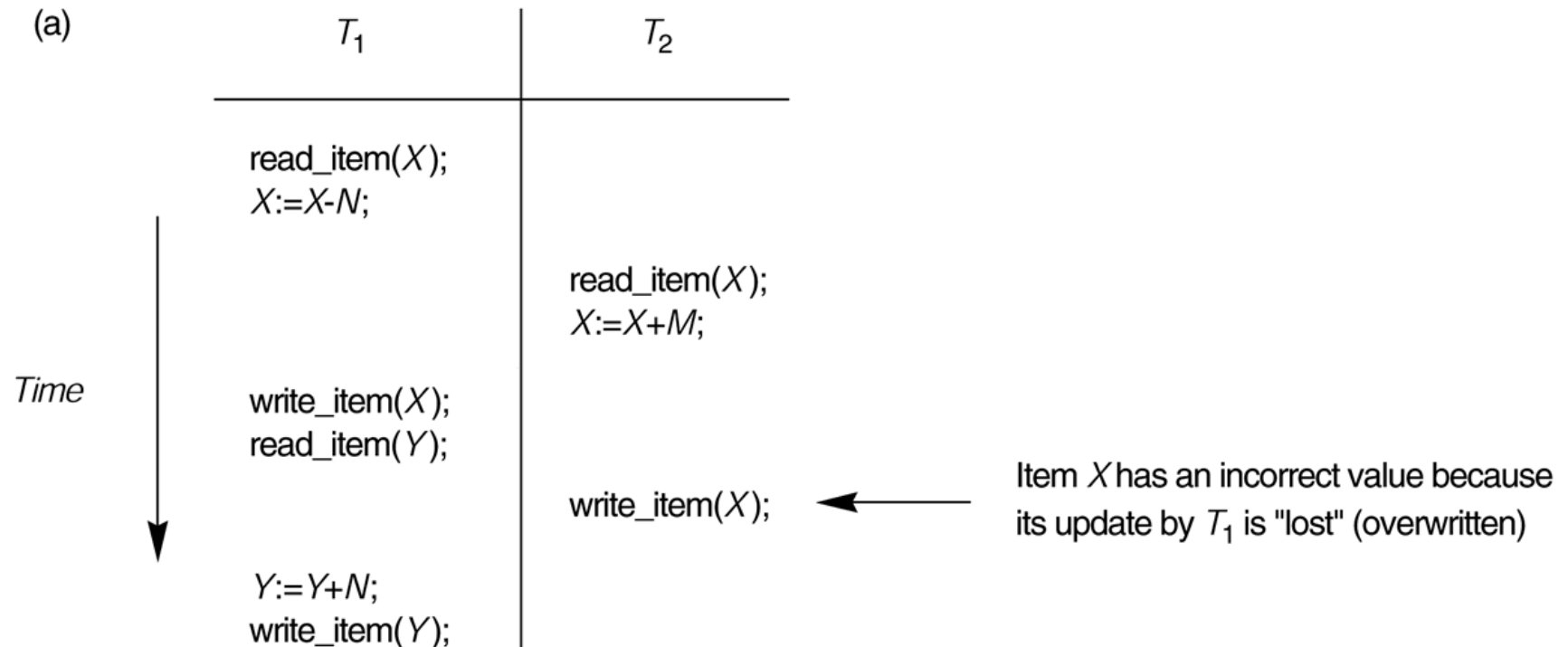


Why Concurrency Control is needed:

1. The Lost Update Problem.

This occurs when two transactions that access the same database items have their operations interleaved in a way that makes the value of some database item incorrect.

Some problems that occur when concurrent execution is uncontrolled. (a) The lost update problem.

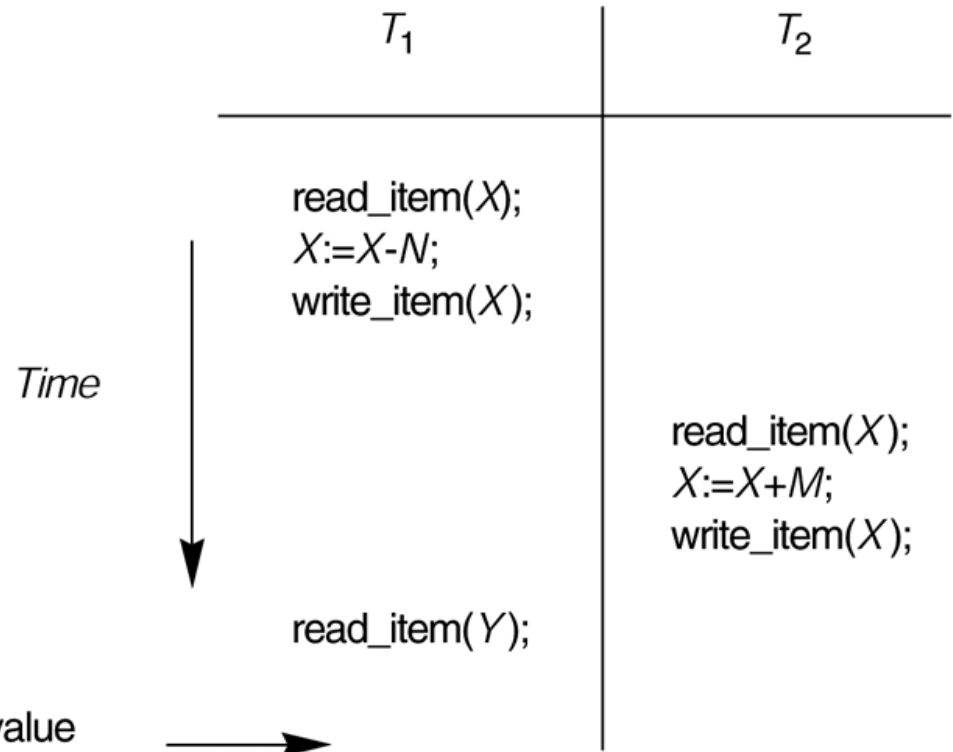


2. The Temporary Update (or Dirty Read) Problem.

This occurs when one transaction updates a database item and then the transaction fails for some reason . The updated item is accessed by another transaction before it is changed back to its original value.

Some problems that occur when concurrent execution is uncontrolled. (b) The temporary update problem.

(b)

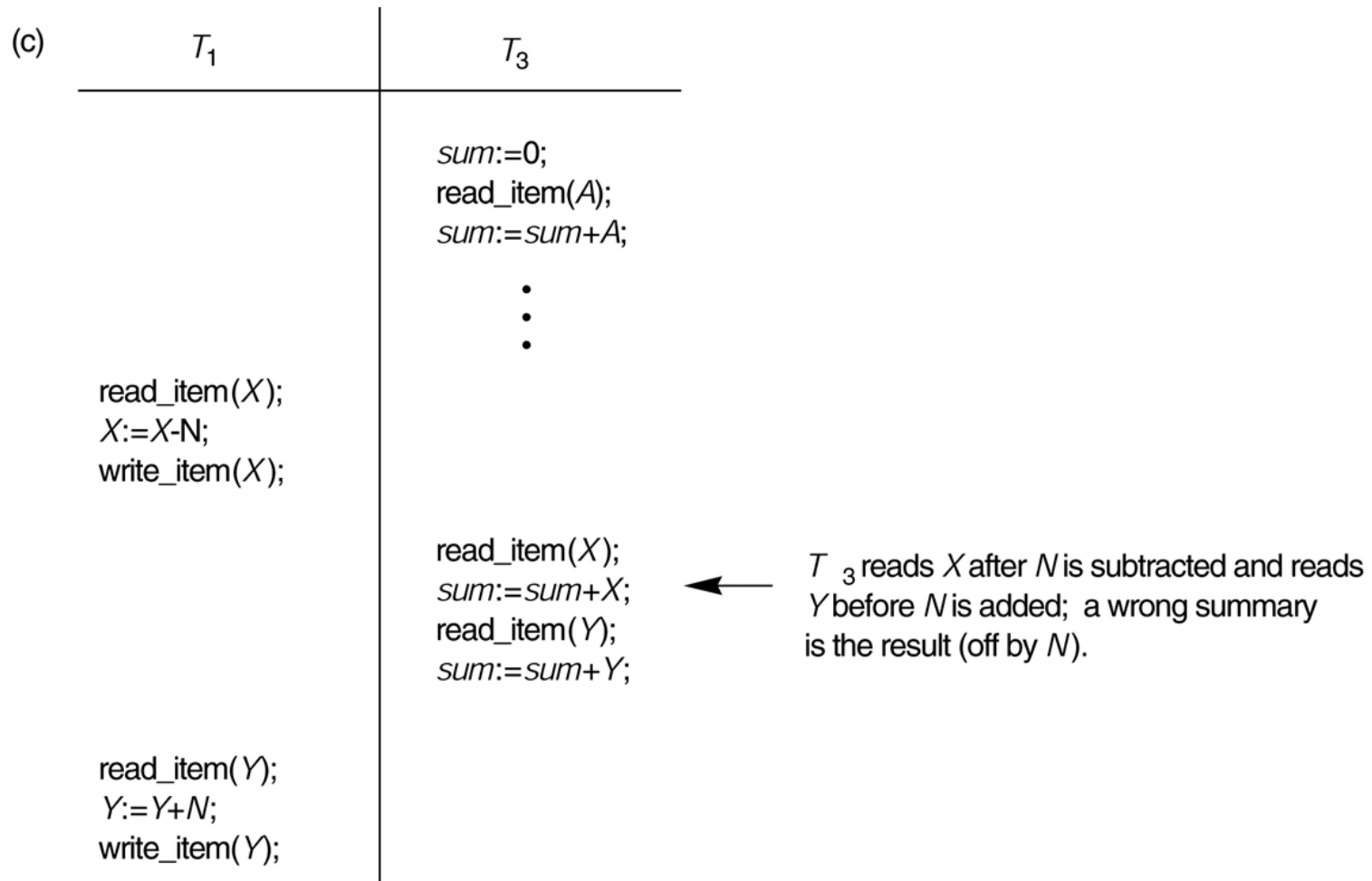


Transaction T_1 fails and must change the value of X back to its old value; meanwhile T_2 has read the "temporary" incorrect value of X .

3. The Incorrect Summary Problem .

If one transaction is calculating an aggregate summary function on a number of records while other transactions are updating some of these records, the aggregate function may calculate some values before they are updated and others after they are updated.

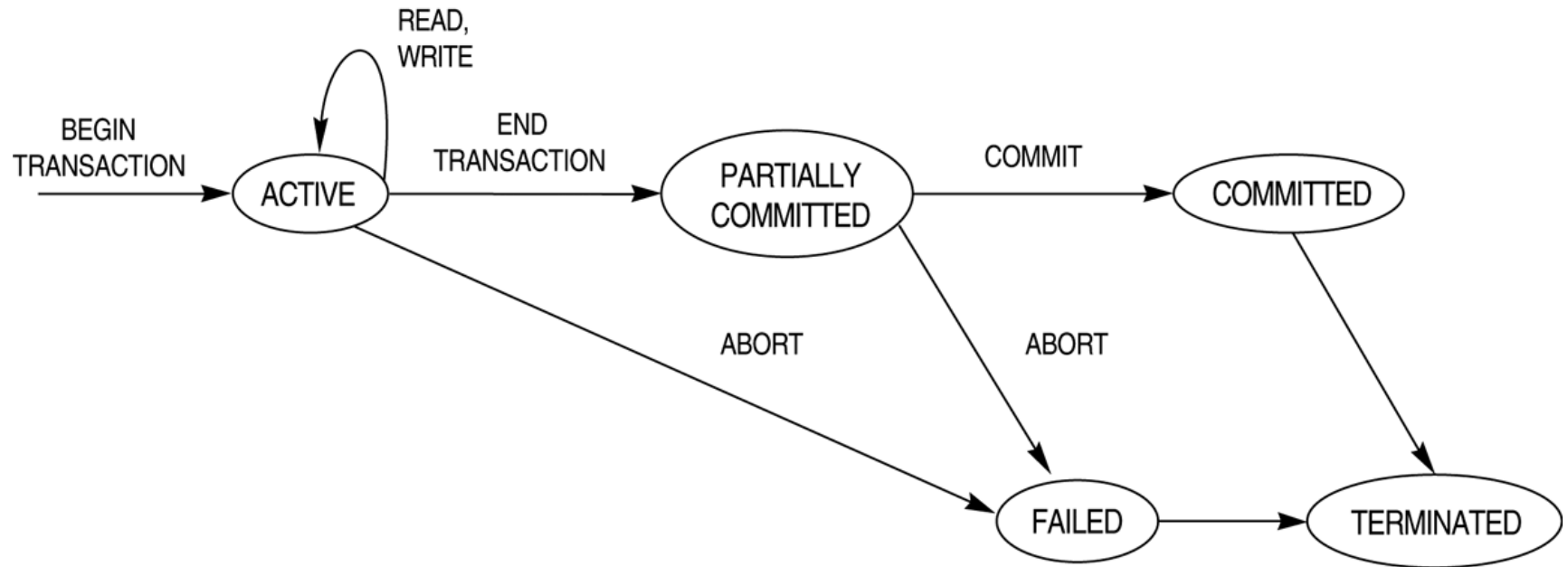
Some problems that occur when concurrent execution is uncontrolled. (c) The incorrect summary problem.



What causes a Transaction to fail

1. **A computer failure (system crash)**
2. **A transaction or system error**
3. **Local errors or exception conditions** detected by the transaction
4. **Concurrency control enforcement**
5. **Disk failure**
6. **Physical problems and catastrophes**

State transition diagram illustrating the states for transaction execution.



Transaction and System Concepts

A **transaction** is an atomic unit of work that is either completed in its entirety or not done at all

Transaction states:

- Active state
- Partially committed state
- Committed state
- Failed state
- Terminated State

Transaction and System Concepts

Recovery manager keeps track of the following operations:

- **begin_transaction**
- **read or write**
- **end_transaction**
- **commit_transaction**
- **rollback (or abort)**

Transaction and System Concepts

Recovery techniques use the following operators:

- **undo:** the effect of these write operations of a transaction T by tracing backward through the log and resetting all items changed by a write operation of T to their old_values.
- **redo:** This specifies that certain *transaction operations* must be *redone* to ensure that all the operations of a committed transaction have been applied successfully to the database.

Transaction and System Concepts

Commit Point of a Transaction:

- **Definition:** A transaction T reaches its **commit point** when all its operations that access the database have been executed successfully *and* the effect of all the transaction operations on the database has been recorded in the log. Beyond the commit point, the transaction is said to be **committed**, and its effect is assumed to be *permanently recorded* in the database. The transaction then writes an entry [commit,T] into the log.
- **Roll Back of transactions:** Needed for transactions that have a [start_transaction,T] entry into the log but no commit entry [commit,T] into the log.

Transaction and System Concepts

Commit Point of a Transaction (cont):

- **Redoing transactions:** Transactions that have written their commit entry in the log must also have recorded all their write operations in the log; otherwise they would not be committed, so their effect on the database can be *redone* from the log entries.
- **Force writing a log:** *before* a transaction reaches its commit point, any portion of the log that has not been written to the disk yet must now be written to the disk. This process is called force-writing the log file before committing a transaction.

Types of log record:

1. **[start_transaction,T]**: Records that transaction T has started execution.
2. **[write_item,T,X,old_value,new_value]**: Records that transaction T has changed the value of database item X from old_value to new_value.
3. **[read_item,T,X]**: Records that transaction T has read the value of database item X.
4. **[commit,T]**: Records that transaction T has completed successfully, and affirms that its effect can be committed (recorded permanently) to the database.
5. **[abort,T]**: Records that transaction T has been aborted.

Desirable Properties of Transactions

ACID properties:

- **Atomicity:** A transaction is an atomic unit of processing; it is either performed in its entirety or not performed at all.
- **Consistency preservation:** A correct execution of the transaction must take the database from one consistent state to another.

Desirable Properties of Transactions

ACID properties (cont.):

- **Isolation:** A transaction should not make its updates visible to other transactions until it is committed; this property, when enforced strictly, solves the temporary update problem and makes cascading rollbacks of transactions unnecessary
- **Durability or permanency:** Once a transaction changes the database and the changes are committed, these changes must never be lost because of subsequent failure.

Characterizing Schedules based on Recoverability

- **Transaction schedule or history:** When transactions are executing concurrently in an interleaved fashion, the order of execution of operations from the various transactions forms what is known as a transaction schedule (or history).
- A **schedule** (or **history**) S of n transactions $T_1, T_2, \dots, T_n$:
It is an ordering of the operations of the transactions subject to the constraint that, for each transaction T_i that participates in S , the operations of T_i in S must appear in the same order in which they occur in T_i . Note, however, that operations from other transactions T_j can be interleaved with the operations of T_i in S .

schedules

T_1	T_2
read_item(X); $X := X - N$;	
	read_item(X); $X := X + M$;
write_item(X); read_item(Y);	
$Y := Y + N$; write_item(Y);	write_item(X);

$S_a: r_1(X); r_2(X); w_1(X); r_1(Y); w_2(X); w_1(Y);$

Schedules

- Two operations are said to be Conflict if they satisfy all of the three following conditions:
 - ▣ They belong to different transactions
 - ▣ They access the same item X
 - ▣ At least one of the operation is a write_item(X)

Conflict operations:	r1(X) and w2(X) w1(X) and w2(X)
----------------------	------------------------------------

Non-Conflict operations:	r1(X) and r2(X)
--------------------------	-----------------

Characterizing Schedules based on Recoverability

Schedules classified on recoverability:

- **Recoverable schedule:** One where no *committed* transaction needs to be rolled back.

A schedule S is **recoverable** if no transaction T in S commits until all transactions T' that have written an item that T reads have committed.

- **Cascadeless schedule:** One where every transaction reads only the items that are written by committed transactions.

Schedules requiring cascaded rollback: A schedule in which uncommitted transactions that read an item from a failed transaction must be rolled back.

Characterizing Schedules based on Recoverability

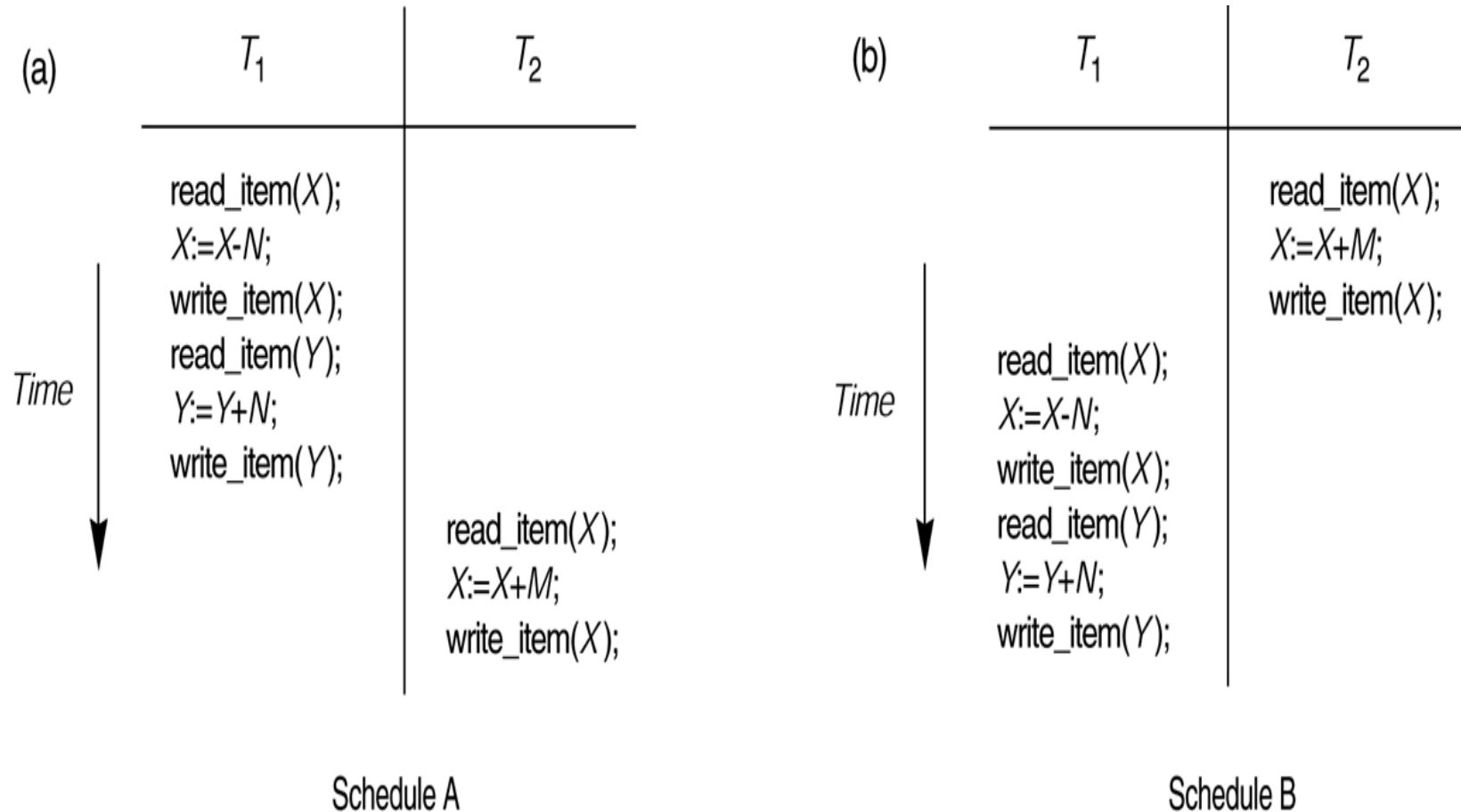
Schedules classified on recoverability (cont.):

- **Strict Schedules:** A schedule in which a transaction can neither read or write an item X until the last transaction that wrote X has committed.

Characterizing Schedules based on Serializability

- **Serial schedule:** A schedule S is **serial** if, for every transaction T participating in the schedule, all the operations of T are executed consecutively in the schedule. Otherwise, the schedule is called **nonserial schedule**.
- **Serializable schedule:** A schedule S is **serializable** if it is equivalent to some serial schedule of the same n transactions.

Serial schedule



Characterizing Schedules based on Serializability

- **Result equivalent:** Two schedules are called result equivalent if they produce the same final state of the database.
- **Conflict equivalent:** Two schedules are said to be conflict equivalent if the order of any two conflicting operations is the same in both schedules.
- **Conflict serializable:** A schedule S is said to be conflict serializable if it is conflict equivalent to some serial schedule S' .

Two schedules that are result equivalent for the initial value of $X = 100$ but are not result equivalent in general.

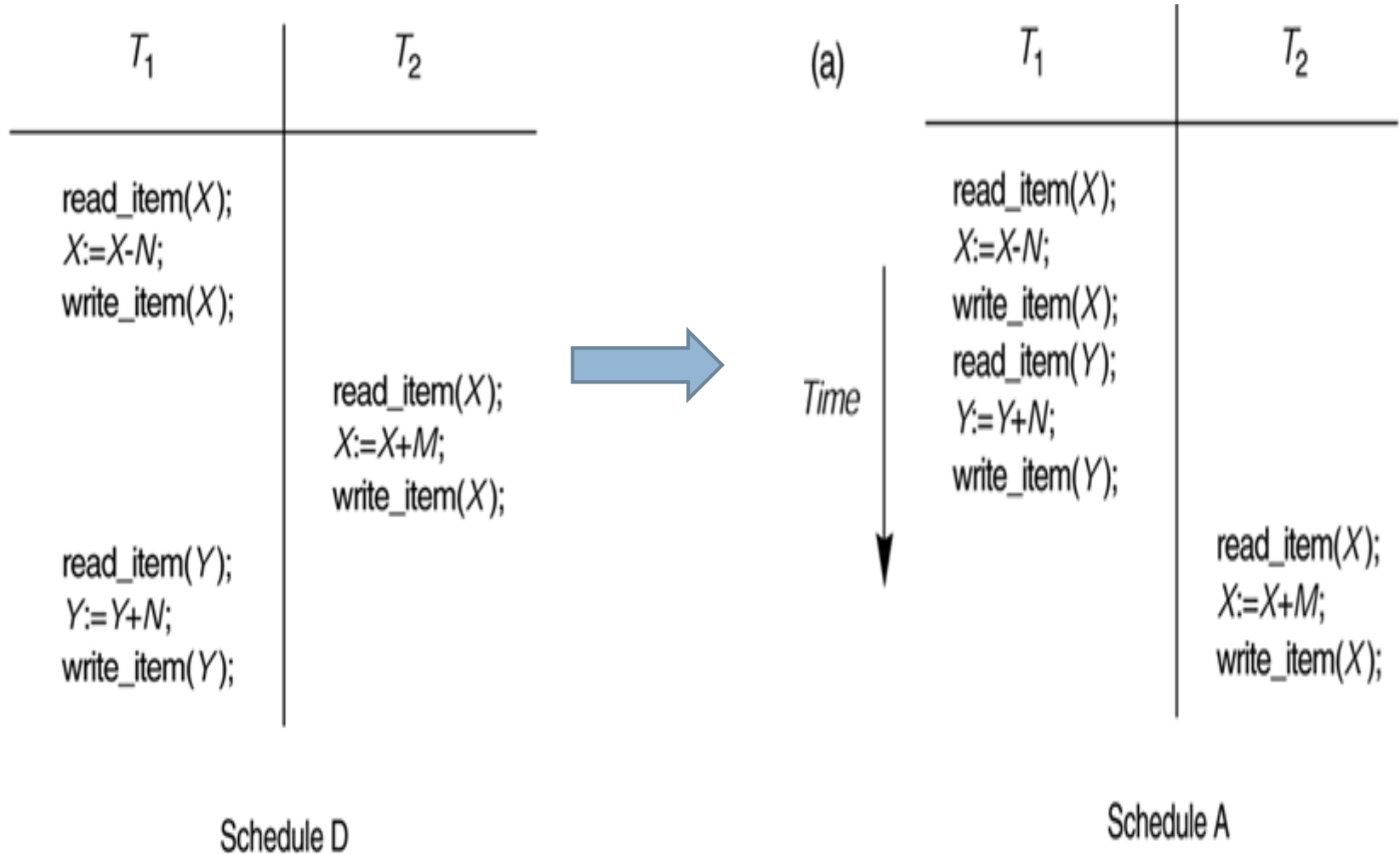
S_1

read_item(X);
 $X := X + 10$;
write_item(X);

S_2

read_item(X);
 $X := X * 1.1$;
write_item(X);

Conflict serializable



Characterizing Schedules based on Serializability

- Being serializable is not the same as being serial
- Being serializable implies that the schedule is a correct schedule.
 - ▣ It will leave the database in a consistent state.
 - ▣ The interleaving is appropriate and will result in a state as if the transactions were serially executed, yet will achieve efficiency due to concurrent execution.

Characterizing Schedules based on Serializability

- Serializability is hard to check.
 - ▣ Interleaving of operations occurs in an operating system through some scheduler
 - ▣ Difficult to determine beforehand how the operations in a schedule will be interleaved.

Characterizing Schedules based on Serializability

Practical approach:

- Come up with methods (protocols) to ensure serializability.
- It's not possible to determine when a schedule begins and when it ends. Hence, we reduce the problem of checking the whole schedule to checking only a *committed project* of the schedule (i.e. operations from only the committed transactions.)
- Current approach used in most DBMSs:
 - ▣ Use of locks with two phase locking

Characterizing Schedules based on Serializability (6)

- **View equivalence:** A less restrictive definition of equivalence of schedules
- **View serializability:** definition of serializability based on view equivalence. A schedule is *view serializable* if it is *view equivalent* to a serial schedule.

Characterizing Schedules based on Serializability (7)

Two schedules are said to be **view equivalent** if the following three conditions hold:

1. The same set of transactions participates in S and S' , and S and S' include the same operations of those transactions.
2. For any operation $R_i(X)$ of T_i in S , if the value of X read by the operation has been written by an operation $W_j(X)$ of T_j (or if it is the original value of X before the schedule started), the same condition must hold for the value of X read by operation $R_i(X)$ of T_i in S' .
3. If the operation $W_k(Y)$ of T_k is the last operation to write item Y in S , then $W_k(Y)$ of T_k must also be the last operation to write item Y in S' .

Characterizing Schedules based on Serializability (8)

The premise behind view equivalence:

- As long as each read operation of a transaction reads the result of *the same write operation* in both schedules, the write operations of each transaction must produce the same results.
- **“The view”**: the read operations are said to see the *the same view* in both schedules.

Characterizing Schedules based on Serializability (9)

Relationship between view and conflict equivalence:

- The two are same under **constrained write assumption** which assumes that if T writes X, it is constrained by the value of X it read; i.e., $\text{new } X = f(\text{old } X)$
- Conflict serializability is **stricter** than view serializability. With unconstrained write (or blind write), a schedule that is view serializable is not necessarily conflict serializable.
- Any conflict serializable schedule is also view serializable, but not vice versa.

Characterizing Schedules based on Serializability (10)

Relationship between view and conflict equivalence (cont):

Consider the following schedule of three transactions

T1: $r_1(X)$, $w_1(X)$; T2: $w_2(X)$; and T3: $w_3(X)$:

Schedule Sa: $r_1(X)$; $w_2(X)$; $w_1(X)$; $w_3(X)$; c_1 ; c_2 ; c_3 ;

In Sa, the operations $w_2(X)$ and $w_3(X)$ are blind writes, since T2 and T3 do not read the value of X.

Sa is **view serializable**, since it is view equivalent to the serial schedule T1, T2, T3. However, Sa is **not conflict serializable**, since it is not conflict equivalent to any serial schedule.

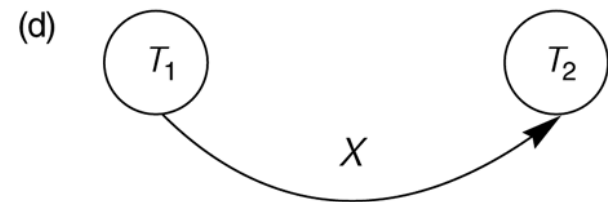
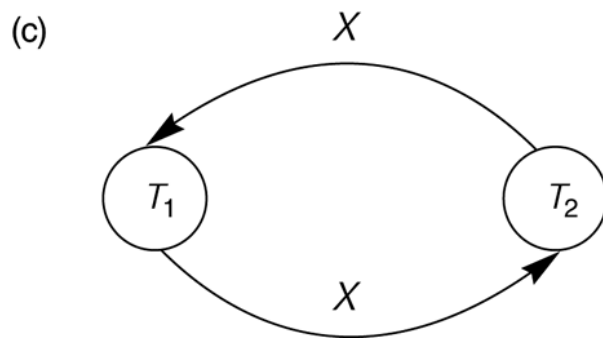
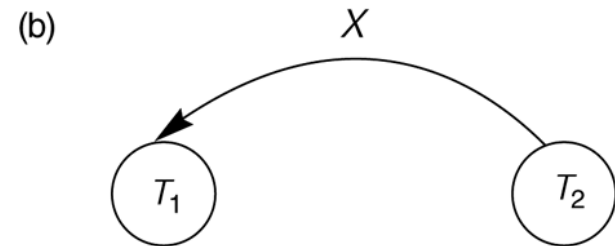
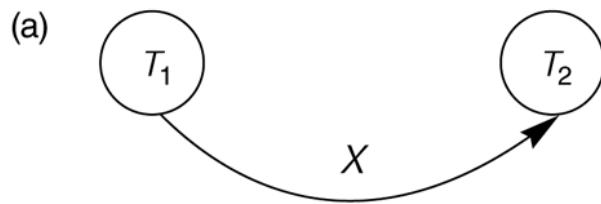
Characterizing Schedules based on Serializability (11)

Testing for conflict serializability

Algorithm 17.1:

1. Looks at only read_Item (X) and write_Item (X) operations
2. Constructs a precedence graph (serialization graph) - a graph with directed edges
3. An edge is created from T_i to T_j if one of the operations in T_i appears before a conflicting operation in T_j
4. The schedule is serializable if and only if the precedence graph has no cycles.

Constructing the precedence graphs for schedules A and D from Figure 17.5 to test for conflict serializability. (a) Precedence graph for serial schedule A . (b) Precedence graph for serial schedule B . (c) Precedence graph for schedule C (not serializable). (d) Precedence graph for schedule D (serializable, equivalent to schedule A).



Another example of serializability testing. (a) The READ and WRITE operations of three transactions T_1 , T_2 , and T_3 .

(a)

transaction T_1
read_item (X); write_item (X); read_item (Y); write_item (Y);

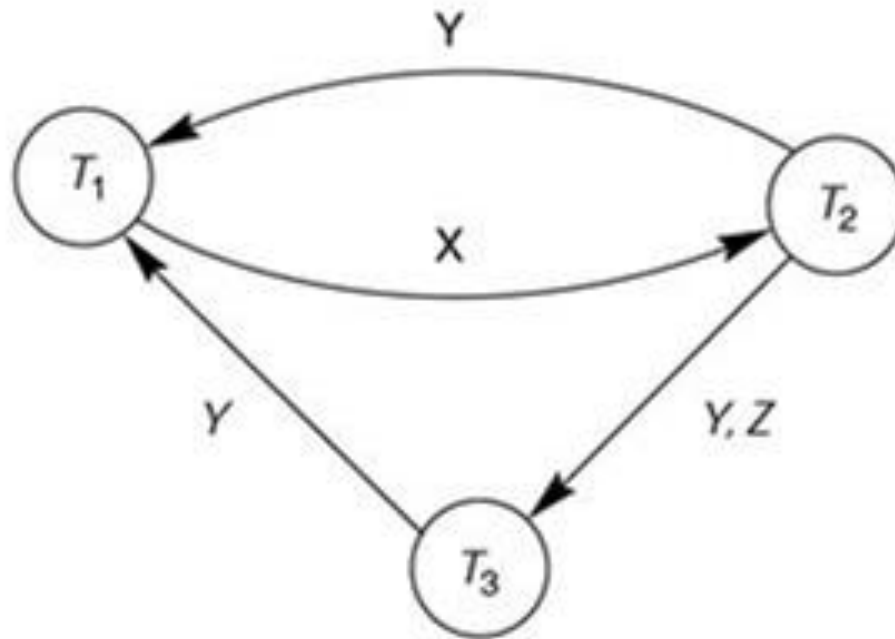
transaction T_2
read_item (Z); read_item (Y); write_item (Y); read_item (X); write_item (X);

transaction T_3
read_item (Y); read_item (Z); write_item (Y); write_item (Z);

Another example of serializability testing. (b) Schedule *E*.

(b)	transaction T_1	transaction T_2	transaction T_3
Time ↓	read_item (X); write_item (X);	read_item (Z); read_item (Y); write_item (Y);	read_item (Y); read_item (Z);
	read_item (Y); write_item (Y);	read_item (X);	write_item (Y); write_item (Z);
		write_item (X);	

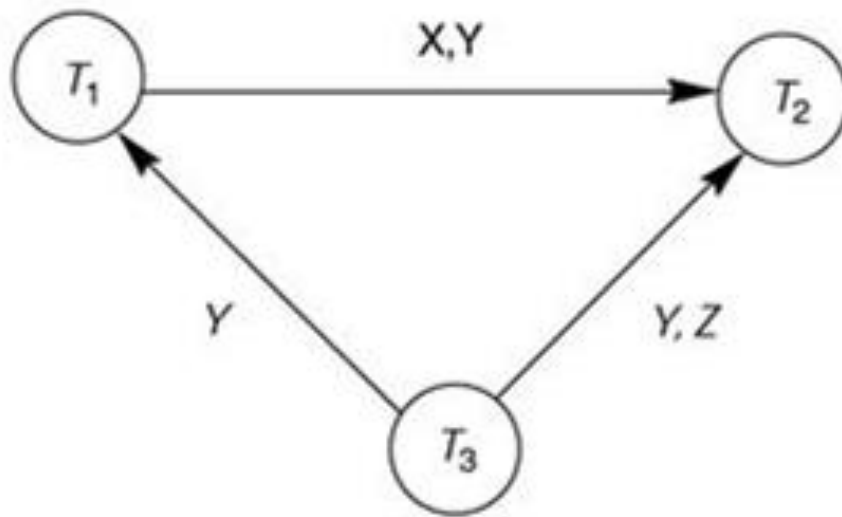
Schedule E



Another example of serializability testing. (c) Schedule *F*.

(c)	transaction T_1	transaction T_2	transaction T_3
<i>Time</i> 	read_item (X); write_item (X);	read_item (Z);	read_item (Y); read_item (Z);
	read_item (Y); write_item (Y);	read_item (Y); write_item (Y); read_item (X); write_item (X);	write_item (Y); write_item (Z);

Schedule F



Equivalent serial schedules

$T_3 \rightarrow T_1 \rightarrow T_2$